

Risk-Aware Reinforcement Learning with Bandit-Based Adaptation for Quadrupedal Locomotion

Yiqun Song

Contribution

- We propose a framework for learning risk-aware policies via CVaR-constrained RL.
- We present an online UCB based multi-armed bandit that selects the risk level online to adapt to unknown environment conditions.
- We conduct comprehensive evaluation in simulation and on a Unitree Go2 Edu robot, demonstrating robust behavior under diverse disturbances and rapid convergence of the selection mechanism to the best policy.

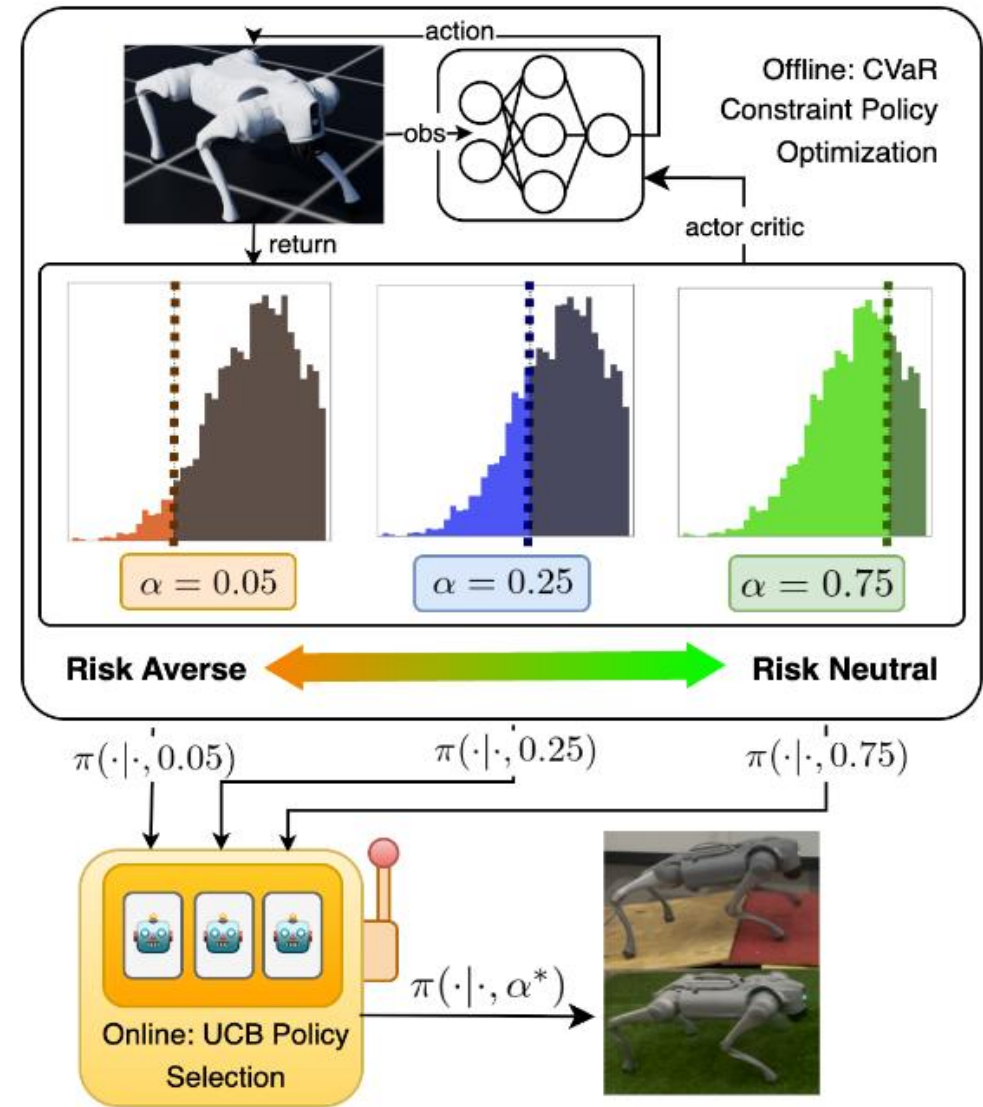


Fig. 1. Overview: We train multiple policies using CVaR-constrained policy optimization, where each critic focuses on a different tail of the return distribution, resulting in policies with varying levels of risk awareness. An upper confidence bound (UCB) bandit is then used to adaptively select the appropriate risk level online. The bandit selects the best performing policy over time as the robot interacts with the environment repeatedly.

Mean or Risk ?

- Plan A: $P(\text{Earn } 100)=90\%$, $P(\text{Lose } 50)=10\%$, $E[\text{Earn}]=76$
- Plan B: $P(\text{Earn } 80)=95\%$, $P(\text{Lose } 10)=5\%$, $E[\text{Earn}]=75.5$
- Which one would you choose ?
- We are more concerned about how much we would lose in the worst-case.

Value At Risk

- For random variable R , confidence level $\alpha \in (0,1)$, we have

$$\text{VaR}_\alpha(R) := \inf\{\eta \mid P(R \leq \eta) \geq \alpha\}$$

which is the inverse function of $\text{CDF}(R)$.

- Not convex: local extrema in Gradient Descent.
- Only threshold, but not how bad it will be.

Conditional Value At Risk

- For random variable R , confidence level $\alpha \in (0,1)$, we have **convex**

$$\text{CVaR}_\alpha(R) := \mathbb{E}[R \mid R \leq \text{VaR}_\alpha(R)]$$

which is the conditional expectation of all returns below the VaR threshold.

- By **Rockafellar-Uryasev Theorem**, we have

$$\text{CVaR}_\alpha(R) := \sup_{\eta \in \mathbb{R}} \left\{ \eta - \frac{1}{\alpha} \mathbb{E}[(\eta - R)^+] \right\}$$

- Risk-Neutral: $\alpha \rightarrow 1$, $\text{CVaR} \rightarrow \mathbb{E}[R]$
- Risk-Averse: $\alpha \rightarrow 0$, $\text{CVaR} \rightarrow \inf R$
- CVaR provides a convex lower bound of VaR:

$$0 \leq \text{CVaR}_\alpha(R) \leq \text{VaR}_\alpha(R) \implies \mathbb{P}(R \leq 0) \leq \alpha.$$

CVaR Constrained PPO

- Previous work: CVaR as PPO target

$$\max_{\theta} \text{CVaR}_{\alpha} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

- Low sample utilization: high-performing samples are ignored.
- Training oscillation when α is small.

CVaR Constrained PPO

- Our work: CVaR as constraint

$$\max_{\theta} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad \text{s.t. } \text{CVaR}_{\alpha}(R) \geq \beta$$

- With Lagrangian relaxation,

$$\max_{\lambda \geq 0} \min_{\theta, \eta} \mathcal{L}(\theta, \eta, \lambda) := -J(\pi_{\theta}) + \lambda \left[\beta - \eta + \frac{1}{\alpha} \mathbb{E}_{\xi \sim \pi_{\theta}} [(\eta - D(\xi))^+] \right]$$

$$\nabla_{\theta} \mathcal{L} = -\mathbb{E}_{\xi \sim \pi_{\theta}} \left[\nabla_{\theta} \log P_{\theta}(\xi) \left(D(\xi) - \frac{\lambda}{\alpha} (\eta - D(\xi))^+ \right) \right]$$

CVaR Constrained PPO

- With clipped surrogate similar to PPO

$$\begin{aligned}\mathcal{L}_{\text{PPO-Lagrangian}} := & -\mathbb{E}_{(s,a) \sim \pi_{\theta_{\text{old}}}} [\min(rA(s,a), g(\epsilon, A(s,a)))] \\ & + \frac{\lambda}{\alpha} \mathbb{E}_{\xi_T \sim \pi_{\theta_{\text{old}}}} \left[\min \left(r(\eta - \tilde{D}(\xi_T))^+, g(\delta, (\eta - \tilde{D}(\xi_T))^+) \right) \right]\end{aligned}$$

- Where $g(\epsilon, A)$ is PPO clipped function

$$g(\epsilon, A) = \begin{cases} (1 + \epsilon)A & A \geq 0 \\ (1 - \epsilon)A & A < 0 \end{cases}$$

- And bootstrap returns

$$\tilde{D}(\xi_T) = \sum_{t=0}^{T-1} \gamma^t R(s_t, a_t) + \gamma^T V_{M, \pi_{\theta_{\text{old}}}}(s_T)$$

Algorithm 1 Offline: CVaR Policy Optimization

Require: hyperparameters $\alpha, \epsilon, \delta, \lambda_{\max}$ learning rates $lr_{\eta}, lr_{\theta}, lr_{\lambda}$, function f for updating β , rollout length T

Ensure: policy π_{θ}

```
1: for  $j = 1, 2, \dots, N_{\text{iter}}$  do
2:   Generate  $N$  trajectories  $\{\xi_T^{(i)}\}_{i=1}^N$  with  $\pi_{\theta_{\text{old}}}$ .
3:   Compute  $A(s_t, a_t)$  and  $\tilde{D}(\xi_T^{(i)}) \forall i, \forall (s_t, a_t) \in \xi_T^{(i)}$ 
4:   Calculate  $\nabla_{\theta} L_{\text{PPO-Lagrangian}}$ 
5:    $\eta \leftarrow \eta - lr_{\eta} \cdot \nabla_{\eta} L$ 
6:    $\theta \leftarrow \theta - lr_{\theta} \cdot \nabla_{\theta} L_{\text{PPO-Lagrangian}}$ 
7:    $\lambda \leftarrow \lambda + lr_{\lambda} \cdot \nabla_{\lambda} L$ 
8:    $\lambda \leftarrow \min(\max(0, \lambda), \lambda_{\max})$ 
9:    $\beta \leftarrow f(\{\xi_T^{(i)}\}_{i=1}^N)$ 
10:   $\theta_{\text{old}} \leftarrow \theta$ 
11: end for
```

Deployment

- After offline training, we got several policy $\pi(\cdot|\alpha)$ which one to use?
- Low Distortion Env $\rightarrow$ Risk Neutral $\rightarrow$ Large α
- High Distortion Env $\rightarrow$ Risk Averse $\rightarrow$ Small α
- A multi-armed bandits is used to choose policy.

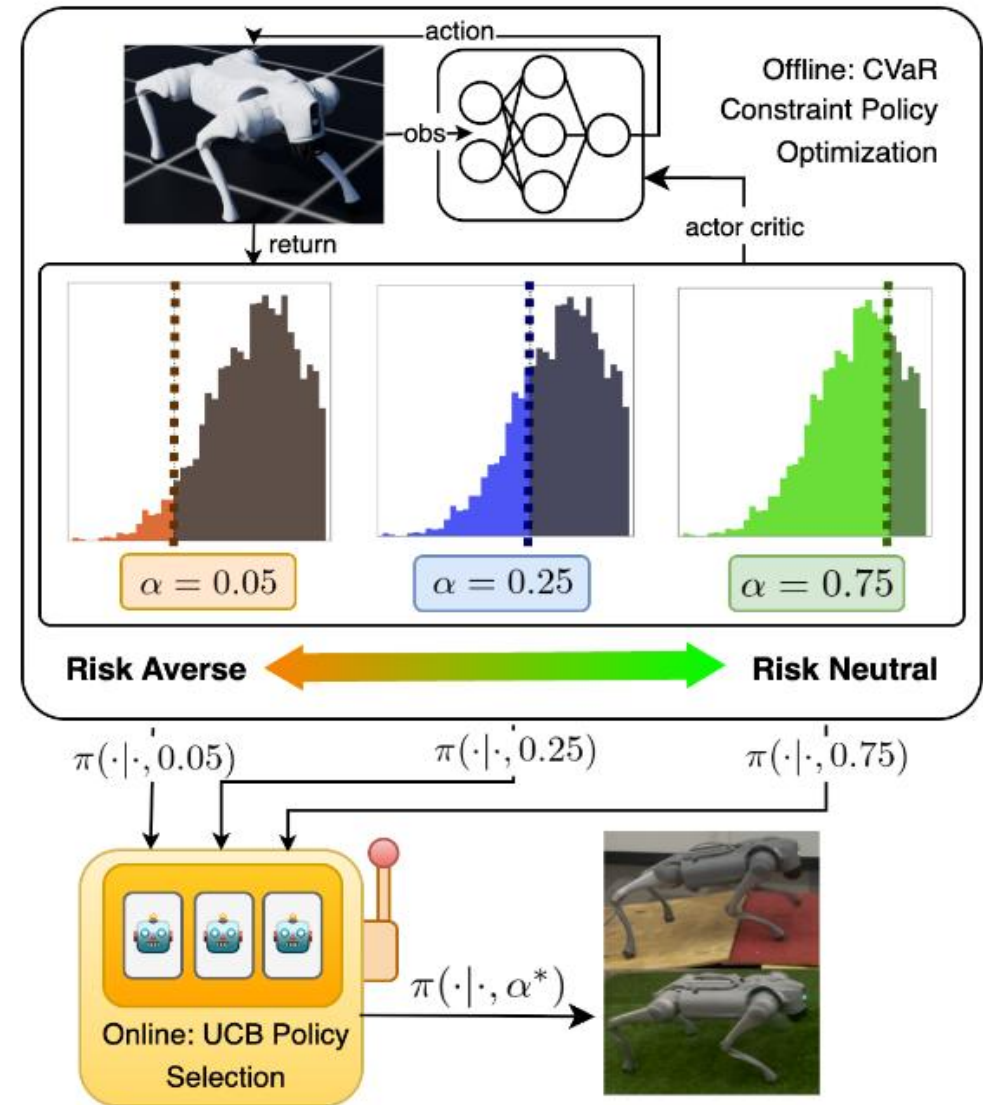
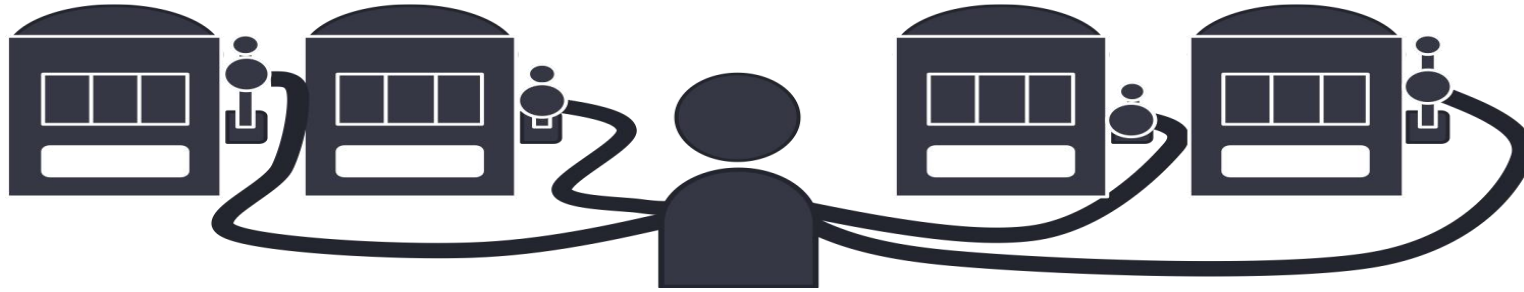


Fig. 1. Overview: We train multiple policies using CVaR-constrained policy optimization, where each critic focuses on a different tail of the return distribution, resulting in policies with varying levels of risk awareness. An upper confidence bound (UCB) bandit is then used to adaptively select the appropriate risk level online. The bandit selects the best performing policy over time as the robot interacts with the environment repeatedly.

Multi-Armed Bandits

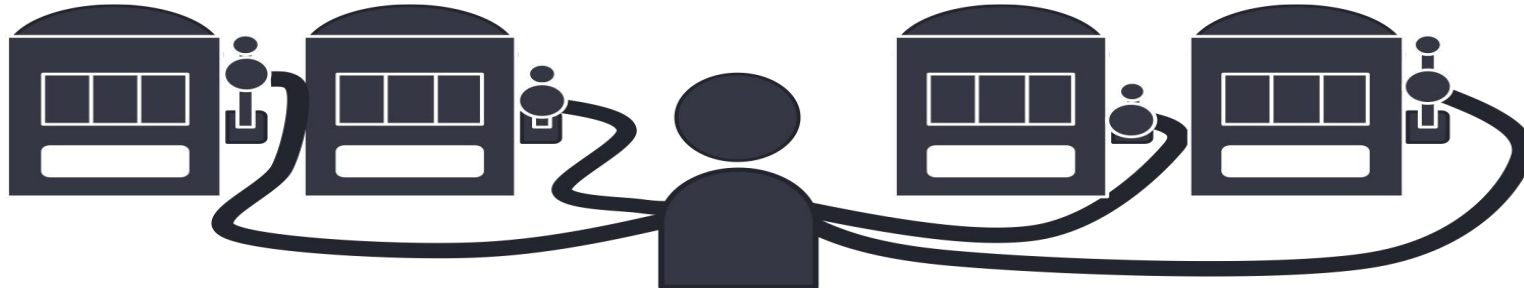


- As known in Sutton's RL book, we have a K-armed bandit. Each arm has a reward distribution that we cannot observe.
- We select the k-th arm and got a reward each time, and our target is to maximize total reward. A natural idea is calculate

$$\hat{\mu}_k = \frac{\sum_{i=1}^{t-1} R_i \cdot [\text{choose k-th arm}]}{\sum_{i=1}^{t-1} [\text{choose k-th arm}]}$$

- And select the max $\hat{\mu}_k$ arm to pull.

Multi-Armed Bandits

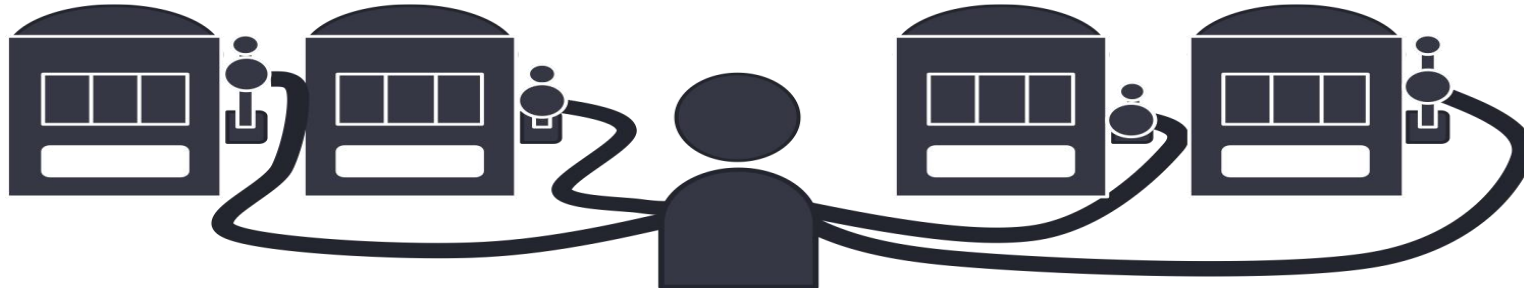


- But how about exploration ?
- We assume the real μ_k has confidence bound $(\hat{\mu}_k - \delta, \hat{\mu}_k + \delta)$, with **Hoeffding's Inequality**

$$\mathbb{P}(|\hat{\mu}_k - \mu_k| \leq \delta) \geq 1 - 2 \exp\left(-\frac{2N_k\delta^2}{(b-a)^2}\right) = 1 - \frac{2}{t^c} \quad \text{with } \delta = c\sqrt{\frac{\log t}{N_k}}$$

- And **UCB bound** $\text{UCB}_k(t) = \hat{\mu}_k + c\sqrt{\frac{\log t}{N_k}}$

Multi-Armed Bandits



- We have policy group $\{\pi_{\theta_k}(\cdot|\cdot, \alpha_k)\}_{k=1}^K$, each policy is a bandit arm.
- Every episode e , we choose k -th policy and got a trajectory with undiscounted return $X_{k,e}$. And estimate(privileged information is lacked)

$$\hat{\mu}_k(e) = \frac{1}{N_k(e)} \sum_{l=1}^e X_{k,l}$$

- The UCB Bound is used to choose policy.

Empirical-Bernstein UCB

- We use mean value as UCB bound, but how about variance?
- The **Bernstein's Inequality** gives a tighter bound

$$\text{UCB}_k(e) := \hat{\mu}_k(e) \sqrt{\frac{2 \hat{\sigma}_k(e) \ln(3/\delta_e)}{N_k(e)}} + \frac{3R \ln(3/\delta_e)}{N_k(e)}$$

- And $\hat{\sigma}_k(e) = \frac{1}{N_k(e)-1} \sum_{l=1}^e (X_{k,l} - \hat{\mu}_k(e))^2$ to estimate the variance.

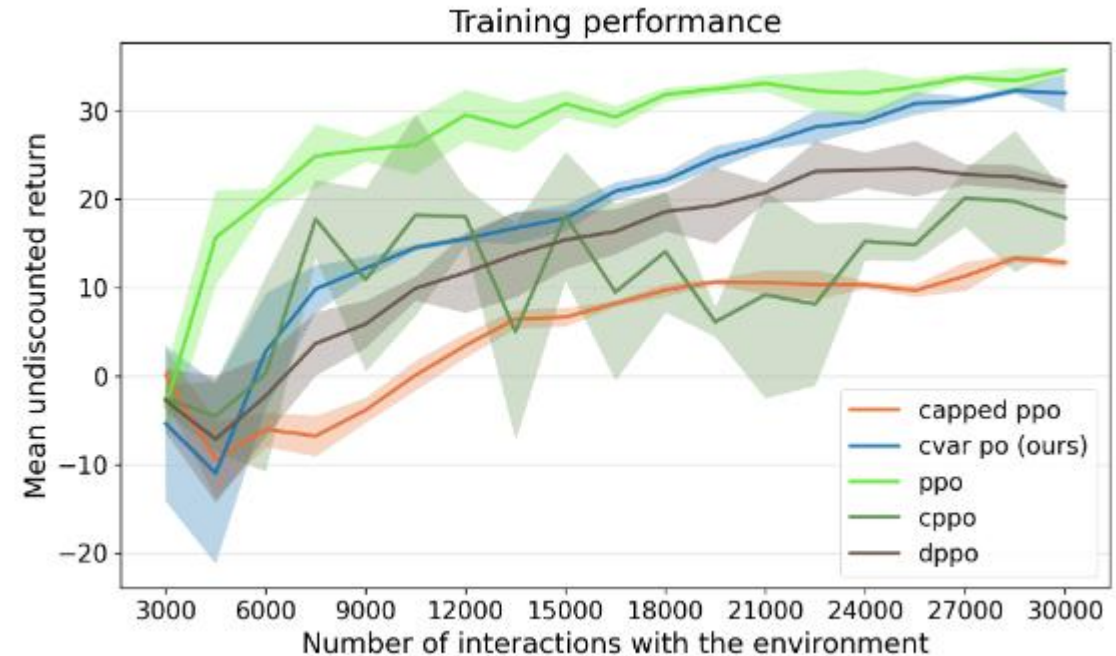
Algorithm 2 Online: Empirical-Bernstein UCB

Require: policies with different α -level $\{\pi_{\theta_k}(\cdot|\cdot, \alpha_k)\}_{k=1}^K$, reward range R , episode length T

- 1: Initialize selection counter $N_k \leftarrow 0$
- 2: **for** $k = 1$ to K **do**
- 3: Run π_{θ_k} for T step; save episodic return x
- 4: Calculate $\hat{\mu}_k$ and $\hat{\sigma}_k$ from history of x
- 5: **end for**
- 6: **for** $e = K + 1$ to E **do**
- 7: $\delta_e \leftarrow \delta / (K e^2)$
- 8: **for** $k = 1$ to K **do**
- 9: Calculate $\hat{\sigma}_k$ and $\hat{\mu}_k$ from history of x
- 10: $U_k \leftarrow \text{UCB}(\hat{\mu}_k, \hat{\sigma}_k, N_k, R, \delta_e)$
- 11: **end for**
- 12: $k^* \leftarrow \arg \max_k U_k$
- 13: $N_{k^*} \leftarrow N_{k^*} + 1$
- 14: Run policy $\pi_{\theta_{k^*}}$ for T step; save episodic return x
- 15: **end for**

Algorithm Performance

- Training performance at $\alpha = 0.25$
- Our method learns slowly at the start, due to optimizing the Lagrangian term, but attains comparable performance as PPO at the end.
- CPPO is unstable and slower to converge, motivating the clipping and bootstrapping components in our design.
- Capped PPO and DPPO, which both directly target the CVaR objective, display similar behavior for much of training.



Robustness Under Disturbances

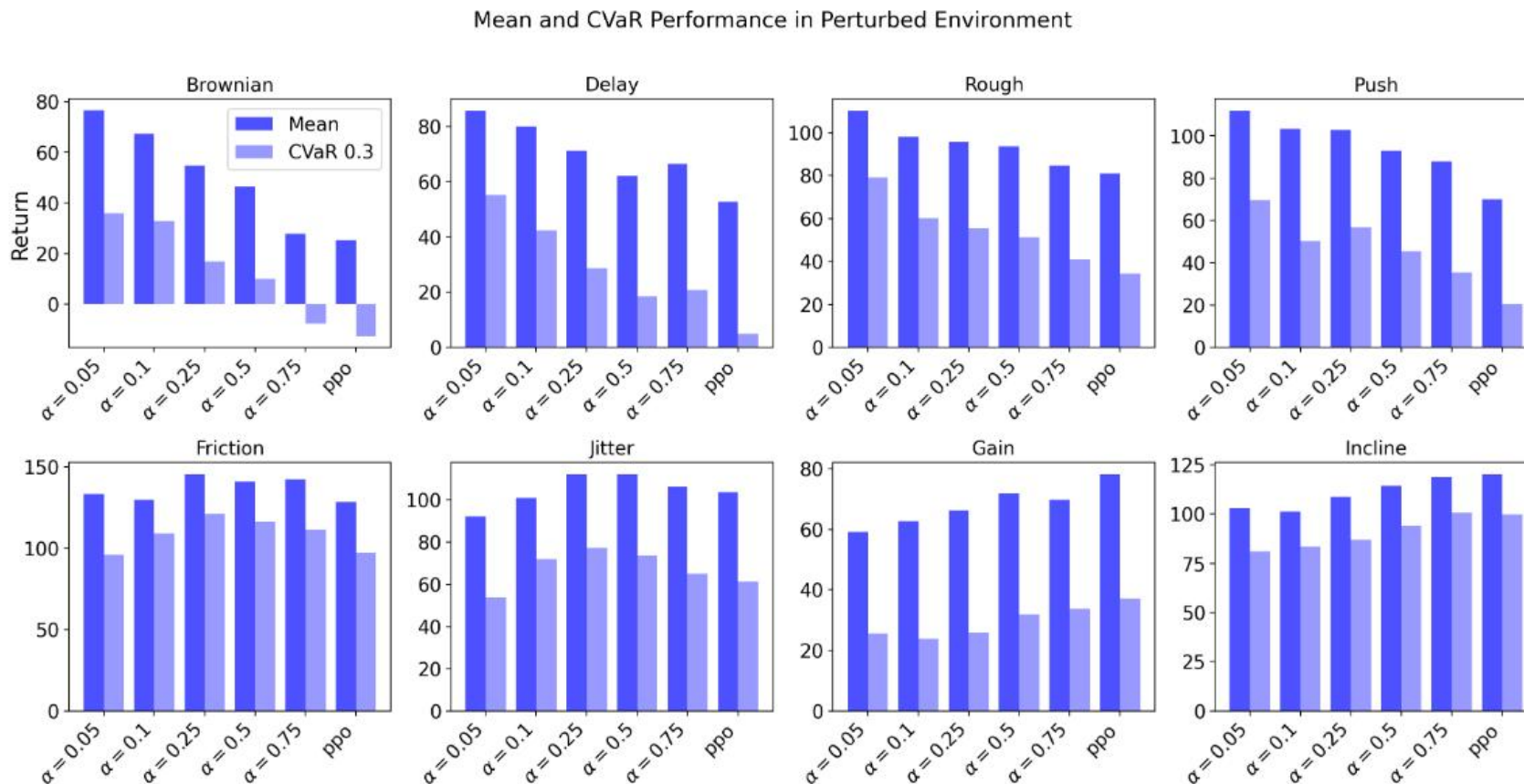


Fig. 3. Performance of with different α -level. We trained policies different α -level and test their performances in perturbed environments. The descriptions of the environments are available at Table III. For each α -level we trained 10 policies using different seeds, and we evaluate each policy with 1000 parallel simulations and report the mean return over 6000 timesteps. We reset the environment and randomize the state of robots every 100 timesteps

Sim-to-real Robustness

TABLE IV

POLICY PERFORMANCE ACROSS THREE ENVIRONMENTS.

Experiment	Policy	Time/TTL (s)	Succ (%)	Reward/sec
Flat	$\alpha = 0.05$	18.0 ± 5.0	100	1.05 ± 0.20
	$\alpha = 0.25$	15.0 ± 7.0	100	1.31 ± 0.25
	PPO	12.0 ± 6.0	100	1.35 ± 0.30
Ramp	$\alpha = 0.05$	19.0 ± 8.0	55	0.95 ± 0.35
	$\alpha = 0.25$	12.5 ± 6.4	70	1.32 ± 0.32
	PPO	7.4 ± 6.3	30	1.18 ± 0.46
Grass	$\alpha = 0.05$	77.0 ± 9.4	85	1.01 ± 0.13
	$\alpha = 0.25$	59.2 ± 28.1	60	1.14 ± 0.32
	PPO	64.2 ± 36.0	55	1.09 ± 0.44

